Your Course Code - Course Title

Cue: Real-Time Desktop AI Copilot

Screen-Aware Assistance Through a Floating Desktop Interface

1024170400 Samyak Jain
1024170388 Sangam Sangwan

BE Third Year, CSE

Project Advisor

Ms Paramveer Sidhu

Project Progress Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **2**

1.1 Background and Context 2

1.1.1 Problem Statement 2

1.2 Project Objectives 2

2 Methodology and Design **4**

2.1 System Architecture 4

2.1.1 High-Level Design 4

2.1.2 Technical Stack 4

2.2 Implementation Details 5

2.2.1 Module 1: Desktop Interface and Screenshot Capture 5

2.2.2 Module 2: On-Screen Assistance 5

2.2.3 Project Progress 5

3 Results and Evaluation **6**

3.1 Testing Methodology 6

3.2 Performance Metrics 6

3.3 Analysis 7

4 Conclusion **8**

4.1 Summary of Work 8

4.2 Key Findings 8

4.3 Future Work and Recommendations 8

4.4 Final Remarks 9

Chapter 1

Introduction

1.1 Background and Context

Modern desktop work often requires users to switch between their active application and separate AI tools to ask questions about on-screen information. During coding, studying, or working with documents, manually explaining the visible context creates friction and interrupts concentration.

Cue is a desktop AI copilot designed to make assistance accessible through a floating interface. The current implementation focuses on desktop integration, screenshot capture, and asking questions about on-screen content. The longer-term goal is to extend this screen context with voice and audio from calls so that the assistant can provide more context-aware help during conversations.

1.1.1 Problem Statement

Traditional chat-based assistants generally rely on information that users manually type or upload. They may not have direct access to the context currently visible on the user's desktop. This makes it harder to ask questions about a screen, code snippet, or application without changing windows or copying information.

The project addresses this problem by building a desktop application that can stay accessible while the user works and provide assistance based on captured screen content. Future work will extend the context available to the assistant to include voice and audio from calls.

1.2 Project Objectives

1. **Desktop accessibility:** Build a floating desktop interface that can be invoked while the user continues working in other applications.

2. **Screenshot capture:** Capture the current screen when needed to provide visual context to an AI model.

3. **On-screen assistance:** Allow users to ask questions about visible content and receive context-aware responses.

4. **Screen-share testing:** Evaluate the application's visibility behavior during video calls and screen recording on the team's laptops.

5. **Voice and call context:** Extend the application to listen to relevant voice and video-call audio and use that context in subsequent assistance.

Chapter 2

Methodology and Design

2.1 System Architecture

Cue is implemented as a desktop application. Its architecture combines a floating user interface, desktop integration, screenshot capture, and communication with an AI model. The current milestone is centered on screenshot-based and on-screen support; voice and video-call listening and context integration remain the next stage of development.

2.1.1 High-Level Design

The current workflow is as follows:

1. The user opens Cue and invokes the assistant from its desktop interface.
2. When screen context is required, the application captures a screenshot.
3. The user submits a question or request related to the visible content.
4. The screenshot and request are used to construct an AI prompt.
5. The response is displayed through the Cue interface.

The next stage is to add voice and video-call listening context so that relevant conversation audio can be transcribed or otherwise processed and incorporated into requests. This work is not presented as completed in the current progress report.

2.1.2 Technical Stack

- **Desktop framework:** Electron.
- **Application language:** JavaScript and Node.js.
- **Screen capture:** Electron desktop-capture functionality.
- **AI integration:** Configurable AI model/provider integration used for screen-aware responses.

- **Development and testing:** Local development and testing on team laptops, including trials during video calls and screen recording.

2.2 Implementation Details

2.2.1 Module 1: Desktop Interface and Screenshot Capture

The application provides a floating desktop interface intended to remain accessible while the user works in other applications. Screenshot capture supplies visual context for questions about on-screen content. This forms the current functional milestone of the project.

2.2.2 Module 2: On-Screen Assistance

The screenshot-based workflow allows the user to ask questions about content visible on the screen and receive an AI-generated response. The current progress is focused on making this workflow operate end to end, from invoking the interface and capturing the screen to submitting the request and displaying the answer.

2.2.3 Project Progress

The following items describe the current progress without treating planned functionality as already implemented:

- **Completed:** Desktop application and floating interface.
- **Completed:** Screenshot capture and on-screen assistance workflow.
- **Tested:** The application was run on the team's laptops during video calls and while screen recording. In these tests, the Cue window was not visible in the call/recording output.
- **Next:** Add voice and video-call listening so the assistant can use conversation audio as context.

The screen-recording and video-call result above describes the team's observed tests on its laptops. It should not be interpreted as a guarantee that Cue will be hidden from every recording tool, operating system, or capture configuration.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

Testing so far has focused on running the application on the team's own laptops and verifying the screenshot-based desktop workflow. The application was also tested during video calls and while screen recording to check whether the Cue window appeared in the call or recorded output.

- **Functional testing:** Ran the application locally and checked the screenshot and on-screen assistance workflow.
- **Video-call testing:** Ran Cue while participating in video calls and observed whether the Cue window appeared in the call output.
- **Screen-recording testing:** Ran screen recording while Cue was open and checked the resulting recording for visibility of the Cue window.
- **Observed result:** In the team's laptop tests, the Cue window was not visible in the video-call/recording output.
- **Next tests:** Validate voice and video-call listening context once implemented, and repeat the visibility tests across the supported operating systems and capture applications.

These are practical tests performed on the team's laptops. A wider compatibility matrix has not been established in this report, so the observed visibility result should be treated as limited to the tested setups.

3.2 Performance Metrics

The current evaluation is qualitative and focuses on the completed screenshot and on-screen assistance workflow. No verified latency, transcription-accuracy, or throughput measurements are reported at this stage.

Metric	Current Status
Desktop application runs locally	Tested on team laptops
Screenshot and on-screen support	Implemented and tested
Visibility during tested video calls	Cue window not observed in call output
Visibility during tested screen recordings	Cue window not observed in recording output
Voice and call-audio context	Next development milestone
Quantitative performance benchmarks	Not yet measured

Table 3.1: Current implementation and evaluation status

3.3 Analysis

The current milestone establishes the core desktop workflow: Cue can run on the team's laptops, capture screenshot context, and support questions about on-screen content. The team also tested the application's visibility during video calls and screen recording and did not observe the Cue window in those outputs on the tested setups.

The main remaining functional step is to incorporate voice and video-call listening context. Once that capability is implemented, it should be evaluated for transcription or context quality, response usefulness, latency, and platform-specific behavior. The visibility result should also be tested on additional operating systems and capture tools before making broader compatibility claims.

Chapter 4

Conclusion

4.1 Summary of Work

The project has reached the screenshot and on-screen assistance milestone. The desktop application runs on the team's laptops, and its current workflow provides a way to capture screen context and ask questions about visible content. The team has also tested the application during video calls and screen recording; in those tests, the Cue window was not visible in the resulting call or recording output.

4.2 Key Findings

- **Core screen workflow:** Screenshot capture and on-screen assistance are the current completed functional focus.
- **Initial visibility testing:** On the team's tested laptop setups, the Cue window was not observed in video-call or screen-recording output.
- **Next milestone:** Voice and video-call listening context is the next development task and should be evaluated after implementation.

4.3 Future Work and Recommendations

1. Implement voice and video-call listening context.
2. Integrate relevant call audio into the assistant's context and test the end-to-end workflow.
3. Evaluate response usefulness and latency with repeatable scenarios.
4. Repeat screen visibility tests across different operating systems, video-call applications, and screen-recording tools.
5. Document the limitations of capture exclusion and ensure testing and use follow relevant consent and platform requirements.

4.4 Final Remarks

The project has established its initial desktop and screenshot-based assistance workflow. The immediate priority is to build on that foundation by incorporating voice and video-call context, followed by systematic functional and compatibility testing.